

# مشروع: بيئة تطوير ومترجم اللغة العربية (ArabiCompilerIDE)

## أسماء الطلاب:

حسام صادق الشراعي  
عمرو خالد شمس  
أنس عبدالرزاق الصلاحي  
عبدالعزیز مطيع الفهد  
صهيب أنور دماج  
عمرو محمد الحاشدي

اسم المقرر: مترجمات

الفصل الدراسي: الأول

العام الدراسي: 2025/2026

# 1. وصف المشروع والبيانات التقنية

## 1.1. وصف المشروع وكيفية العمل

مشروع **ArabiCompilerIDE** هو بيئة تطوير متكاملة (IDE) ومترجم للغة برمجة عربية. تم تطوير المشروع كتطبيق عملي شامل لمفاهيم تصميم المترجمات، حيث يقوم بتحويل شفرة مصدرية مكتوبة باللغة العربية إلى شفرة هدف بلغة **Assembly (NASM x64)**، ثم يقوم بتجميعها وتنفيذها مباشرة وعرض النتائج بشكل تفاعلي.

**آلية العمل:** تم بناء المشروع بلغة **#C** باستخدام إطار عمل **.NET Framework** وواجهة **WPF**. تم استخدام مكتبة **ANTLR4** لتوليد المحللين اللفظي والنحوي من قواعد اللغة العربية. تمر عملية الترجمة بالمراحل الكلاسيكية (التحليل، توليد الشفرة الوسطية، التحسين) وصولاً إلى توليد شفرة **Assembly**. بعد ذلك، يتم استدعاء أدوات خارجية (**NASM** و **GCC**) لأتمتة عملية التجميع والربط، وتشغيل البرنامج الناتج مع إعادة توجيه الإدخال والإخراج إلى واجهة المستخدم.

## 1.2. لغات البرمجة المستخدمة

| لغة البرمجة            | الإصدار / المعمارية          | الاستخدام في المشروع                                                                        |
|------------------------|------------------------------|---------------------------------------------------------------------------------------------|
| <b>#C</b>              | مستخدم مع .NET Framework 4.8 | اللغة الأساسية لتطوير منطق المترجم وواجهة المستخدم (IDE).                                   |
| <b>Assembly (NASM)</b> | x64                          | اللغة الهدف التي يتم توليد الشفرة بها، مما يسمح بالتحكم الدقيق في الذاكرة والتنفيذ المباشر. |

## 1.3. الأدوات والملفات الجاهزة المستخدمة

| الأداة / المكتبة               | رقم الإصدار | الاستخدام في المشروع                                           |
|--------------------------------|-------------|----------------------------------------------------------------|
| <b>.NET Framework.</b>         | 4.8         | إطار العمل الأساسي لتشغيل تطبيق WPF.                           |
| <b>ANTLR4.Runtime.Standard</b> | 4.13.1      | مكتبة وقت التشغيل للمحلل اللغوي والنحوي المولّد بواسطة ANTLR4. |

| الأداة / المكتبة                | رقم الإصدار   | الاستخدام في المشروع                                                   |
|---------------------------------|---------------|------------------------------------------------------------------------|
| <b>ICSharpCode.AvalonEdit</b>   | 6.3.1.120     | مكون محرر النصوص المتقدم المستخدم في الواجهة، ويدعم تلوين بناء الجملة. |
| <b>NASM (Netwide Assembler)</b> | (إصدار خارجي) | مترجم لغة Assembly لتحويل الشفرة الهدف إلى ملفات Object.               |
| <b>GCC (MinGW-w64)</b>          | (إصدار خارجي) | الرابط (Linker) الذي يدمج ملفات Object مع مكتبات C لإنتاج ملف تنفيذي.  |

## 2. تحليل تفصيلي لأكواد المشروع

في هذا القسم، يتم تحليل الدوال الرئيسية في ملفات المشروع الأساسية عبر جداول توضيحية.

### 2.1. ملف المنسق الرئيسي: `Compiler.cs`

| اسم الدالة             | الوظيفة                                                                               | المدخلات                                     | المخرجات                                                                    |
|------------------------|---------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------|
| <code>Compile</code>   | هي نقطة الدخول ومنسق عملية الترجمة. تستدعي جميع المراحل الستة بالتسلسل وتجمع النتائج. | <code>sourceCode</code> (string)             | <code>CompilerResult</code><br>(كائن يحتوي على نتائج كل المراحل أو الأخطاء) |
| <code>RunLexer</code>  | تنفيذ مرحلة التحليل اللفظي. تحول الكود المصدري إلى تيار من الرموز (Tokens).           | <code>sourceCode</code> (string)             | <code>LexerOutput</code><br>(كائن يحتوي على تيار الرموز وقائمة نصية بها)    |
| <code>RunParser</code> | تنفيذ مرحلة التحليل النحوي. تبني شجرة التحليل (Parse Tree) من تيار الرموز.            | <code>tokenStream</code> (CommonTokenStream) | <code>ProgramContext</code><br>(الجزر الأعلى لشجرة التحليل)                 |

| اسم الدالة                    | الوظيفة                                                                        | المدخلات                                                       | المخرجات                             |
|-------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------|--------------------------------------|
| RunSemanticAnalysis           | تنفيذ مرحلة التحليل الدلالي. تبني جدول الرموز وتتأكد من صحة الأنواع والنطاقات. | parseTree<br>(ProgramContext)                                  | SymbolTable<br>(جدول الرموز المكتمل) |
| RunIntermediateCodeGeneration | تنفيذ مرحلة توليد الشفرة الوسيطة.                                              | parseTree<br>(ProgramContext),<br>symbolTable<br>(SymbolTable) | string (شفرة TAC)<br>(النتيجة)       |
| RunTargetCodeGeneration       | تنفيذ مرحلة توليد الشفرة الهدف.                                                | optimizedCode<br>(string),<br>symbolTable<br>(SymbolTable)     | string (شفرة Assembly)<br>(النتيجة)  |

## 2.2. ملف توليد الشفرة الوسيطة: TACGenerator.cs

| اسم الدالة                | الوظيفة                                                                                                         | المدخلات                                 | المخرجات                                            |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------|-----------------------------------------------------|
| VisitAssignment_statement | تترجم جملة الإسناد (مثل <code>s = 5</code> ) إلى شفرة TAC.                                                      | context<br>(Assignment_statementContext) | null<br>(تضيف الكود مباشرة إلى <code>code_</code> ) |
| VisitOutput_statement     | تترجم جملة الطباعة ( <code>اطبع</code> ) إلى تعليمات <code>param</code> متبوعة بـ <code>call _print, N</code> . | context<br>(Output_statementContext)     | null                                                |

| اسم الدالة                | الوظيفة                                                                                                                    | المدخلات                                 | المخرجات                                                  |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------|-----------------------------------------------------------|
| VisitInput_statement      | تترجم جملة القراءة<br>(اقرأ) إلى<br>تعليمة read.                                                                           | context<br>(Input_statementContext)      | null                                                      |
| VisitWhile_loop_statement | تترجم حلقة<br>طالما إلى<br>شفرة TAC<br>باستخدام ملصقات<br>وقفزات مشروطة.                                                   | context<br>(While_loop_statementContext) | null                                                      |
| VisitExpression           | تترجم التعبيرات<br>الحسابية<br>والعلائقية (مثل<br>أ + ب أو<br>أ < ب) إلى<br>تعليمات TAC<br>وتخزن النتيجة في<br>متغير مؤقت. | context<br>(ExpressionContext)           | string<br>(اسم المتغير<br>المؤقت الذي<br>يحمل<br>النتيجة) |

## 2.3. ملف تحسين الشفرة: Optimizer.cs

| اسم الدالة | الوظيفة                                                                                                | المدخلات                                                           | المخرجات                                                                    |
|------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Optimize   | الوظيفة الرئيسية<br>للمُحسِّن. تقوم بتعريب<br>أسماء المتغيرات<br>(Sanitization) وتحديث<br>جدول الرموز. | intermediateCode (string),<br>originalSymbolTable<br>(SymbolTable) | OptimizationResult (كائن)<br>يحتوي على الكود المحسن<br>وجداول الرموز المحدث |
| Sanitize   | دالة مساعدة تقوم<br>بتحويل اسم متغير<br>عربي (مثل "عداد") إلى<br>اسم فريد بالإنجليزية<br>(مثل "var0"). | name (string)                                                      | string (الاسم المعرب)                                                       |

## 2.4. ملف توليد الشفرة الهدف: TargetCodeGenerator.cs

| اسم الدالة         | الوظيفة                                                                            | المدخلات                                   | المخرجات                                          |
|--------------------|------------------------------------------------------------------------------------|--------------------------------------------|---------------------------------------------------|
| Generate           | الدالة الرئيسية التي تنسق عملية توليد شفرة Assembly الكاملة.                       | optimizedCode (string)                     | string (شفرة Assembly الكاملة)                    |
| AllocateVariable   | دالة مساعدة لحجز مساحة لمتغير على المكس وتخزين إزاحته (offset) بالنسبة للمؤشر rbp. | varName (string)                           | string (تمثيل المتغير في Assembly، مثل [rbp - 8]) |
| TranslateLineToAsm | (دالة داخلية ضمن Generate) تترجم سطرًا واحدًا من TAC إلى تعليمات Assembly مقابلة.  | line (string), stringLiterals (Dictionary) | string (شفرة Assembly جزء من)                     |

## 2.5. ملف التنفيذ التفاعلي: AssemblyRunner.cs

| اسم الدالة                    | الوظيفة                                                                               | المدخلات                                     | المخرجات                    |
|-------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------|
| CompileAndRunInteractiveAsync | تنسق عملية تجميع شفرة Assembly وتنفيذها بشكل تفاعلي، مع إدارة مجاري الإدخال والإخراج. | asmCode (string), Action للتواصل مع الواجهة. | Task (تعمل بشكل غير متزامن) |
| RunProcessAsync               | دالة مساعدة لتشغيل عملية خارجية (مثل NASM أو GCC) وانتظار انتهائها.                   | fileName (string), arguments (string)        | Task (نتيجة تشغيل العملية)  |
| ReadStreamAsync               | تقرأ بشكل غير متزامن من مجرى الإخراج (stdout) أو الأخطاء                              | reader (StreamReader), callback (Action)     | Task                        |

| اسم الدالة | الوظيفة                                                                                         | المدخلات                             | المخرجات          |
|------------|-------------------------------------------------------------------------------------------------|--------------------------------------|-------------------|
|            | (stderr) للبرنامج قيد التنفيذ، سطرًا بسطر.                                                      |                                      |                   |
| SendInput  | ترسل النص الذي يكتبه المستخدم في الواجهة إلى مجرى الإدخال القياسي (stdin) للبرنامج قيد التنفيذ. | process (Process),<br>input (string) | لا يوجد<br>(void) |



### 3. صور شاشات التنفيذ

توضح الصور التالية نتائج تنفيذ الكود المصدري في بيئة التطوير المتكاملة.

#### 3.1. الكود المصدري المنفذ

```
برنامج حساب؛
متغير
س : صحيح؛
ص : صحيح؛
ش : صحيح؛
عداد : صحيح؛
}

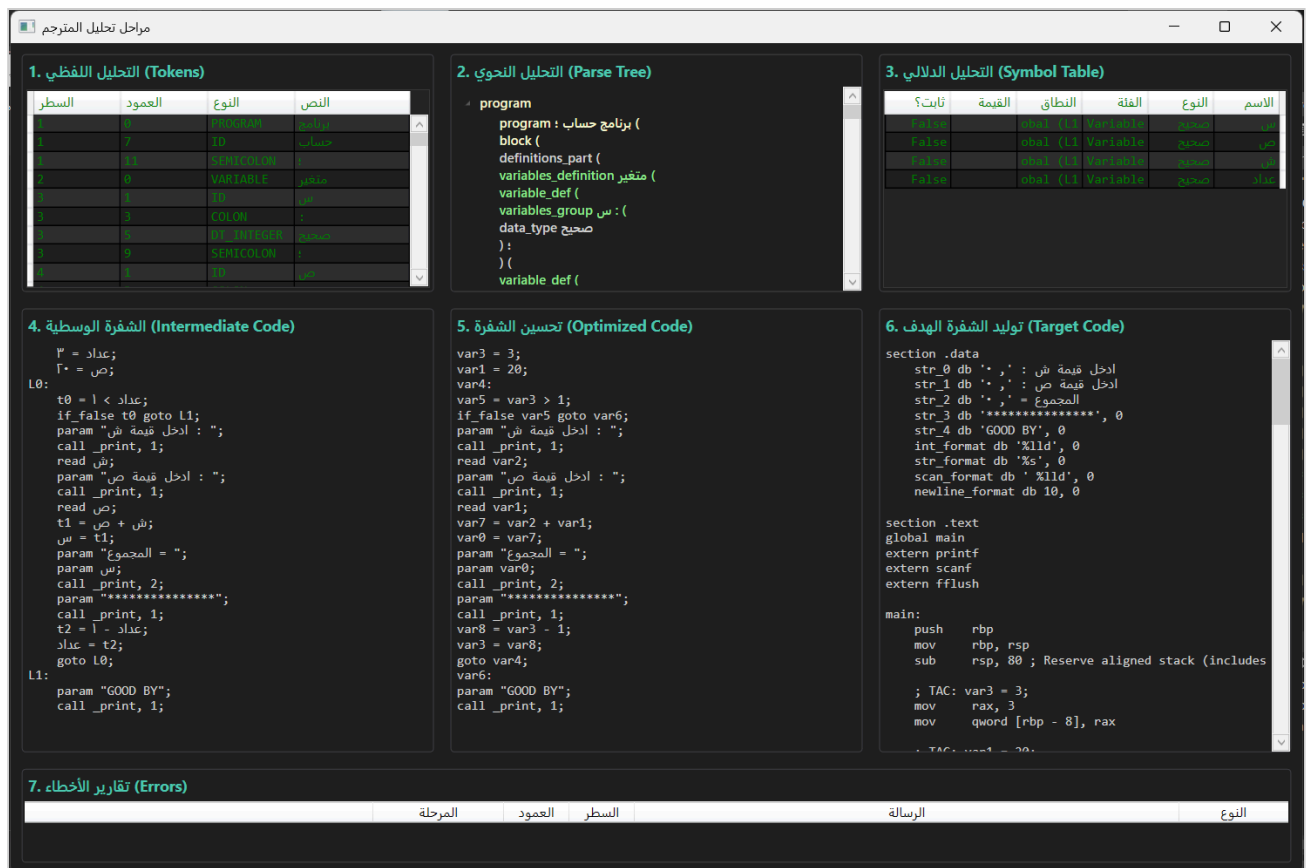
عداد = 3؛
ص = 20؛
طالما (عداد < 1) استمر
}

اطبع ("ادخل قيمة ش : ")؛
اقرا (ش)؛

اطبع ("ادخل قيمة ص : ")؛
اقرا (ص)؛

س = ش + ص؛
اطبع ("المجموع = " , س)؛

اطبع ("*****")؛
عداد = عداد - 1؛
{
اطبع ("GOOD BY")؛
{
.
```



**شكل (1):** نافذة التحليل التي تعرض نتائج المراحل الستة للترجمة، ويظهر فيها بوضوح الشفرة الوسيطة (TAC)، الشفرة المحسنة، والشفرة الهدف النهائية بلغة Assembly.

(مساحة مخصصة لوضع صورة شاشة التنفيذ التفاعلي التي تظهر طلب الإدخال من المستخدم والمخرجات النهائية)

**شكل (2):** الواجهة الرئيسية أثناء التنفيذ التفاعلي. يظهر قسم المخرجات وهو يطلب من المستخدم إدخال قيمة، ثم يعرض النتائج بعد إجراء العمليات الحسابية.

